Phase 3 — Normalization

Campus Complaint & Grievance Tracker — Schema Normalization Case Study

Course: Database Management Systems (DBMS) | **Team:** Chirag Nikant Simepurushkar (24B-CO-015), Raksham R Dessai (24B-CO-019), Aayushi Rajesh Prabhu (24B-CO-001)

1. Overview & Normalization Objectives

The objective of Phase 3 is to mathematically validate the relational schema developed from our Chen-notation Entity-Relationship (ER) model. While an ER diagram captures conceptual entities, cardinalities, and constraints, a naive or denormalized table implementation causes severe data redundancy and anomalies.

In this case study, we start from an unnormalized, flat spreadsheet representation (`All_Complaints_Flat`) and systematically decompose it through **First Normal Form (1NF)**, **Second Normal Form (2NF)**, **Third Normal Form (3NF)**, and **Boyce-Codd Normal Form (BCNF)**. We also prove that the resulting schema satisfies **Lossless-Join Decomposition** and **Dependency Preservation**, reconciling 1:1 with our team's ER model.

2. The Unnormalized Form (UNF) — Flat Spreadsheet

In manual or legacy campus grievance tracking, all operational data is typically dumped into a single flat spreadsheet where each row stores student details, hostel allocation, warden contact details, complaint category, department head, staff assignments, and ongoing status transitions:

```
All_Complaints_Flat (Complaint_ID, Description, Status, Created_At, Updated_At, Category_ID, Category_Name,
SLA_Days, Dept_ID, Dept_Name, Dept_Head, Student_ID, First_Name, Last_Name, Email, Phone_Numbers, Room_No,
Total_Complaints_Filed, Hostel_ID, Hostel_Name, Warden_ID, Warden_First_Name, Warden_Last_Name, Staff_ID,
Staff_First_Name, Staff_Last_Name, Designation, Staff_Email, Assigned_At, Unassigned_At, Status_History_Log)
```

A representative sample of four campus complaints stored in this flat format illustrates the problem:

c_id	Student & Room	Hostel & Warden	Category & Dept	Description	Staff Assigned
101	Raksham Dessai Room 204 | Ph: 9822110011	Nilkanth Hostel Dr. Ramesh Sharma	Hostel (3d SLA) Hostel / Warden Office	Ceiling fan in room 204 making loud grinding noise	John Doe (Tech) `Assigned: Sep 20`
102	Chirag Simepurushkar Room 205 | Ph: 9822110022	Nilkanth Hostel Dr. Ramesh Sharma	Hostel (3d SLA) Hostel / Warden Office	Bathroom tap leaking constantly on 2nd floor	John Doe (Tech) `Assigned: Sep 21`
103	Aayushi Prabhu Room 102 | Ph: 9822110033	Mandovi Hostel Prof. Suresh Naik	Mess (2d SLA) Mess / Catering Dept	Dinner food served cold; water dispenser empty	*Unassigned*
104	Raksham Dessai Room 204 | Ph: 9822110011	Nilkanth Hostel Dr. Ramesh Sharma	Maintenance (5d SLA) Maintenance Dept	Broken study chair leg in room 204	Alex Green (Carp) `Assigned: Sep 15`

Key Observation: *In this single spreadsheet, warden details, department heads, category SLAs, student contact numbers, and staff names are repeatedly duplicated across every complaint row, directly triggering severe database anomalies.*

3. Database Anomalies in the Unnormalized Table

Storing all entities together in the flat spreadsheet above causes three major types of database anomalies that undermine data integrity and administrative reliability:

1. Update / Modification Anomalies

- **Warden Details Duplication:** In the flat table, Dr. Sharma's name and warden ID repeat across complaints 101, 102, 104, and every future complaint filed from Nilkanth Hostel. If the warden of Nilkanth Hostel changes, every historical row referencing Nilkanth Hostel must be located and modified. If an update query misses even one row, conflicting warden records exist.
- **Category SLA & Dept Head Changes:** If the Service Level Agreement for *Hostel* complaints is revised from 3 days to 2 days, or if a new Department Head of Maintenance is appointed, thousands of historical records must be simultaneously modified, causing severe lock contention and risk of partial updates.
- **Student Relocation:** If student Raksham Dessai relocates from room 204 to 210, older records either show outdated information or contradict newer filings.

2. Insertion Anomalies

- **New Student Registration:** A newly admitted student cannot be stored in the database until they submit an initial grievance, because `Complaint_ID` is the primary key of the flat record; storing a student with a `NULL` complaint violates Entity Integrity.
- **New Department or Category Creation:** Administration cannot configure a new campus department (e.g. *Sports Complex*) or category before a student submits a complaint under it.
- **Staff Onboarding:** A newly recruited maintenance technician or hostel warden cannot be entered into the system until an active complaint is assigned to them.

3. Deletion Anomalies

- **Accidental Student Record Erasure:** If complaint 103 is resolved and purged, all record of student Aayushi Prabhu is permanently and unintentionally erased from the university database.
- **Department/Category Loss:** If all complaints under *Mess* are archived or deleted, the fact that the Mess Catering Department exists and has an SLA of 2 days is erased from the operational database.
- **Staff Assignment History Loss:** Deleting closed complaints eliminates the historical log of staff workload and resolution performance.

Conclusion of Anomaly Analysis: *The presence of update, insertion, and deletion anomalies confirms that the flat design is functionally unacceptable for a campus tracking system. Normalization is essential to establish independent, well-bounded relations.*

4. Formal Functional Dependency (FD) Analysis

A Functional Dependency $X \rightarrow Y$ holds over relation R if and only if any two tuples having identical values for attribute set X must also have identical values for attribute set Y. Based on our ER model, the complete set of non-trivial Functional Dependencies is established:

Relation / Context	Determinant (LHS)	Determined Attributes (RHS)
FD1 (Student)	`Student_ID`	First_Name, Last_Name, Email, Room_No, Hostel_ID
FD2 (Student_Phone)	`Student_ID, Phone_Number`	*(All-key relation; determines full tuple)*
FD3 (Staff)	`Staff_ID`	First_Name, Last_Name, Designation, Email, Dept_ID
FD4 (Department)	`Dept_ID`	Dept_Name, Dept_Head
FD5 (Department Candidate)	`Dept_Name`	Dept_ID, Dept_Head
FD6 (Category)	`Category_ID`	Category_Name, SLA_Days, Dept_ID
FD7 (Hostel)	`Hostel_ID`	Hostel_Name, Warden_ID
FD8 (Complaint Core)	`Complaint_ID`	Description, Status, Created_At, Updated_At, Student_ID, Category_ID
FD9 (StatusHistory)	`Complaint_ID, History_Num`	Old_Status, New_Status, Changed_At
FD10 (Staff_Assignment)	`Staff_ID, Complaint_ID, Assigned_At`	Unassigned_At

Transitive Dependencies Deliberately Avoided in the Schema

In the flat unnormalized table, attributes depend on non-candidate keys through multiple indirect paths:

- **Department Transitivity:** `Complaint_ID → Category_ID → Dept_ID → {Dept_Name, Dept_Head}`
- **Hostel & Warden Transitivity:** `Complaint_ID → Student_ID → Hostel_ID → {Hostel_Name, Warden_ID}`

Key Architecture Verification: *As highlighted in our team's ER document, Complaint intentionally does* ***NOT*** *store a* `Hostel_ID` *directly. Because a complaint's hostel can always be derived via* `Complaint → Student → Hostel`*, storing* `Hostel_ID` *directly inside Complaint introduces an unnecessary transitive dependency and violates 3NF.*

5. Step-by-Step Normalization Process

Step 1: First Normal Form (1NF)

Rule: All attribute values must be atomic (indivisible single values; no repeating groups, arrays, or composite multi-valued columns).

- **Decomposition of Multivalued Attributes:** In the ER model, `Phone_Numbers` is multivalued (double ellipse). In UNF, multiple phone numbers stored in a single cell violate atomicity. We decompose this by extracting it into an independent table: `Student_Phone(Student_ID, Phone_Number)`.
- **Decomposition of Composite Attributes:** The composite attribute `Name` on Student and Staff is broken down into atomic components: `First_Name` and `Last_Name`.
- **Elimination of Derived Attributes:** `Total_Complaints_Filed` is a derived attribute (dashed ellipse); it is omitted from physical tables and computed dynamically via `COUNT(*)`.
- **Elimination of Repeating History Logs:** `Status_History_Log` and reassignments are decomposed into discrete records, with composite key `{Complaint_ID, Staff_ID, History_Num, Assigned_At}`.

Step 2: Second Normal Form (2NF)

Rule: The relation must be in 1NF, and every non-prime attribute must be *fully functionally dependent* on the complete candidate key (no partial functional dependencies on a proper subset of a composite key).

- **Identification of Partial Dependencies in 1NF:**
  - `{Complaint_ID} → {Description, Status, Student_ID, Category_ID, Created_At, Updated_At}` depends solely on `Complaint_ID`, independent of `Staff_ID` or `History_Num` (Partial Dependency).
  - Assignment details depend on `{Staff_ID, Complaint_ID, Assigned_At}`.
  - Status change details depend on `{Complaint_ID, History_Num}`.
  - Student, Staff, Department, and Hostel attributes depend only on their respective IDs.
- **Decomposition into 2NF:** Decomposing by determinant produces relations with single-attribute primary keys: `COMPLAINT_BASE`, `STAFF_ASSIGNMENT`, `STATUS_HISTORY`, `STUDENT`, `STAFF`, `HOSTEL`, and `CATEGORY_DEPT`.

Step 3: Third Normal Form (3NF)

Rule: The relation must be in 2NF, and no non-prime attribute may be transitively dependent on the primary key (no chains $X \to Y$ and $Y \to Z$). Formally: for every non-trivial functional dependency $X \to A$, either X is a superkey, or A is a prime attribute.

- **Department Details in Category:** In `CATEGORY_DEPT`, `Category_ID → Dept_ID → {Dept_Name, Dept_Head}`. Because `Dept_ID` is not a candidate key of Category, this is transitive.
  Resolution: Decompose into `Department(Dept_ID, Dept_Name, Dept_Head)` and `Category(Category_ID, Category_Name, SLA_Days, Dept_ID)`.
- **Warden Details in Hostel:** In `HOSTEL`, `Hostel_ID → Warden_ID → {First_Name, Last_Name, Designation}`.
  Resolution: Store only `Warden_ID` in Hostel as a foreign key referencing `Staff(Staff_ID)`.

Step 4: Boyce-Codd Normal Form (BCNF)

Rule: A relation is in BCNF if and only if for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a **superkey** (candidate key or primary key).

We test every relation in our decomposed schema against this strict condition:

Relation	Functional Dependencies & Determinants	Determinant Status	BCNF Status
Student	Student_ID → {First_Name, Last_Name, Email, Room_No, Hostel_ID}	Student_ID (PK)	**Meets BCNF**
Student_Phone	(Student_ID, Phone_Number) → Full tuple	Composite Key (PK)	**Meets BCNF**
Staff	Staff_ID → {First_Name, Last_Name, Designation, Email, Dept_ID}	Staff_ID (PK)	**Meets BCNF**
Department	Dept_ID → {Dept_Name, Dept_Head} Dept_Name → {Dept_ID, Dept_Head}	Dept_ID (PK) Dept_Name (Candidate Key)	**Meets BCNF**
Category	Category_ID → {Category_Name, SLA_Days, Dept_ID}	Category_ID (PK)	**Meets BCNF**
Hostel	Hostel_ID → {Hostel_Name, Warden_ID}	Hostel_ID (PK)	**Meets BCNF**
Complaint	Complaint_ID → {Description, Status, Created_At, Updated_At, Student_ID, Category_ID}	Complaint_ID (PK)	**Meets BCNF**
StatusHistory	(Complaint_ID, History_Num) → {Old_Status, New_Status, Changed_At}	Composite Key (PK)	**Meets BCNF**
Staff_Assignment	(Staff_ID, Complaint_ID, Assigned_At) → {Unassigned_At}	Composite Key (PK)	**Meets BCNF**

BCNF Conclusion: *In every single decomposed relation, all determinants (X) are superkeys. Consequently, our entire relational schema satisfies* ***Boyce-Codd Normal Form (BCNF)****, the highest standard of functional normalization.*

6. Mathematical Properties of the Decomposition

1. Lossless-Join Decomposition (Heath's Theorem): A binary decomposition of relation R into R_1 and R_2 is lossless if and only if $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$. In our schema, every child relation links to its parent via an explicit Foreign Key referencing the parent's Primary Key (e.g. $Hostel \cap Student = \{Hostel_ID\}$, where $Hostel_ID \rightarrow Hostel$). Thus, natural joins across relations reconstruct the exact original dataset with zero spurious tuples ($R_1 \bowtie R_2 = R$).

2. Dependency Preservation: A decomposition preserves dependencies if $(F_1 \cup F_2 \cup \ldots \cup F_k)^+ = F^+$. Each dependency FD_1 through FD_{10} operates entirely on attributes stored within a single table:

`{FD1} ⊆ Student, {FD2} ⊆ Student_Phone, {FD3} ⊆ Staff, {FD4, FD5} ⊆ Department, {FD6} ⊆ Category, {FD7} ⊆ Hostel, {FD8} ⊆ Complaint, {FD9} ⊆ StatusHistory, {FD10} ⊆ Staff_Assignment.`

Every dependency is enforced locally upon `INSERT` or `UPDATE` via primary/unique constraints without cross-table joins. Hence, our schema achieves **100% Dependency Preservation**.

7. Cross-Check with the Team's Chen ER Model

We cross-examine the 9 normalized relations against the conceptual entities and relationships established in our team's Chen ER diagram and documentation:

ER Model Construct	Normalized Table	Primary Key	Foreign Keys & References	Normal Form
Student (Entity)	**Student**	`Student_ID`	`Hostel_ID → Hostel.Hostel_ID`	BCNF / 3NF
Phone_Numbers (Multivalued)	**Student_Phone**	`(Student_ID, Phone_Number)`	`Student_ID → Student.Student_ID`	BCNF / 3NF
Staff (Entity)	**Staff**	`Staff_ID`	`Dept_ID → Department.Dept_ID`	BCNF / 3NF
Department (Entity)	**Department**	`Dept_ID`	*None* (Candidate Key: `Dept_Name`)	BCNF / 3NF
Category (Entity)	**Category**	`Category_ID`	`Dept_ID → Department.Dept_ID`	BCNF / 3NF
Hostel (Entity)	**Hostel**	`Hostel_ID`	`Warden_ID → Staff.Staff_ID`	BCNF / 3NF
Complaint (Hub Entity)	**Complaint**	`Complaint_ID`	`Student_ID → Student.Student_ID` `Category_ID → Category.Category_ID`	BCNF / 3NF
StatusHistory (Weak Entity)	**StatusHistory**	`(Complaint_ID, History_Num)`	`Complaint_ID → Complaint.Complaint_ID`	BCNF / 3NF
Assigned_To (M:N Relationship)	**Staff_Assignment**	`(Staff_ID, Complaint_ID, Assigned_At)`	`Staff_ID → Staff.Staff_ID` `Complaint_ID → Complaint.Complaint_ID`	BCNF / 3NF

Architectural Harmony Verified: *The mathematical decomposition performed in Phase 3 aligns 1:1 with the ER constructs developed in Phase 2. The multivalued attribute (`Phone_Numbers`), weak entity (`StatusHistory`), and M:N relationship (`Assigned_To`) are rigorously converted into distinct, fully normalized BCNF tables.*

8. Summary of Phase 3 Deliverables & Hand-Off to Phase 4

- **Unnormalized Relation Documented:** Concrete campus spreadsheet representation with real student, hostel, and grievance records.
- **Anomaly Analysis Completed:** Explicit demonstration of update anomalies (warden contacts, department heads), insertion anomalies (unregistered students, new categories), and deletion anomalies (erasure of student records upon complaint deletion).
- **Functional Dependencies Established:** Full set of 10 FDs covering candidate keys, foreign keys, weak entities, and M:N assignments.
- **Step-by-Step Proof:** Formal progression from UNF → 1NF (atomic domains) → 2NF (elimination of partial dependencies) → 3NF (elimination of transitive dependencies) → BCNF (every determinant is a superkey).
- **Integrity Theorems Proven:** Verified Lossless Join Decomposition (Heath's Theorem) and Full Dependency Preservation.
- **Readiness for Phase 4:** The 9 normalized BCNF tables are fully prepared for physical SQL schema definition (`schema.sql`) with data types, PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints, followed by realistic seed data (`seed_data.sql`).